

◆ Arrays

Um **array** é uma estrutura que guarda vários valores do mesmo tipo em uma sequência ordenada.

Cada valor fica em uma posição chamada **índice**, que começa em 0.

Exemplo: um array de notas pode armazenar várias notas em uma única variável.

Exemplo: `int[] numeros = {10, 20, 30}; Console.WriteLine(numeros[1]);`
→ aqui será impresso o número **20**, que está na posição de índice 1.

◆ Jagged Arrays (Arrays Dentados)

São arrays de arrays. Diferente de uma matriz tradicional, onde todas as linhas têm o mesmo tamanho, no jagged array cada linha pode ter um tamanho diferente.

É útil quando você precisa de uma “tabela” onde cada linha tem quantidades diferentes de dados.

Exemplo: `int[][] jagged = new int[2][]; jagged[0] = new int[] {1, 2}; jagged[1] = new int[] {3, 4, 5}; Console.WriteLine(jagged[1][2]);` → o resultado será **5**, pois estamos acessando o terceiro elemento da segunda linha.

◆ Laços de Repetição

São estruturas usadas para **repetir instruções** automaticamente sem precisar escrever várias vezes.

- **for** → usado quando sabemos quantas vezes queremos repetir.
- **while** → repete enquanto uma condição for verdadeira.
- **do...while** → parecido com o while, mas garante pelo menos uma execução.
- **foreach** → percorre todos os elementos de uma coleção de forma simples.

Exemplo com for: `for(int i=0; i<3; i++){ Console.WriteLine(i); }` → isso imprime os números **0, 1 e 2**.

◆ Recursão

É quando uma função chama a si mesma para resolver um problema.

Muito usada em cálculos matemáticos e em estruturas de dados como árvores.

Exemplos comuns:

- **Fatorial:** o fatorial de um número n é o produto de todos os números de 1 até n .
- **Fibonacci:** é a sequência onde cada número é a soma dos dois anteriores (0, 1, 1, 2, 3, 5, 8...).

Exemplo fatorial: `int Fatorial(int n){ if(n==1) return 1; return n * Fatorial(n-1); }` `Console.WriteLine(Fatorial(5));` → resultado é **120**.

Exemplo Fibonacci: `int Fibonacci(int n){ if(n==0) return 0; if(n==1) return 1; return Fibonacci(n-1) + Fibonacci(n-2); }` `Console.WriteLine(Fibonacci(6));` → resultado é **8**.

◆ Pilhas (Stack)

Uma pilha segue o princípio **LIFO (Last In, First Out)**: o último que entra é o primeiro que sai.

É como uma pilha de pratos: você sempre tira o de cima primeiro.

Usada em desfazer/redo de editores, chamadas de funções etc.

Exemplo: `Stack<int> pilha = new Stack<int>(); pilha.Push(10); pilha.Push(20); Console.WriteLine(pilha.Pop());` → o resultado é **20**, porque foi o último a entrar.

◆ Filas (Queue)

Uma fila segue o princípio **FIFO (First In, First Out)**: o primeiro que entra é o primeiro que sai.

É como uma fila de banco: quem chega primeiro é atendido primeiro.

Usada em sistemas de impressão, filas de processos etc.

Exemplo: `Queue<string> fila = new Queue<string>(); fila.Enqueue("Ana"); fila.Enqueue("Beto"); Console.WriteLine(fila.Dequeue());` → o resultado é **Ana**, que foi o primeiro a entrar.

◆ Listas Ligadas (Linked Lists)

São estruturas onde cada elemento (nó) guarda o valor e uma referência para o próximo nó. Diferente do array, os elementos não ficam juntos na memória, mas “encadeados”. Isso facilita inserir e remover itens no meio da lista.

Exemplo: `LinkedList<string> lista = new LinkedList<string>();
lista.AddLast("A"); lista.AddLast("B"); lista.AddFirst("C");
foreach(var item in lista){ Console.WriteLine(item); }` → será impresso **C, A, B**, pois o "C" foi adicionado no início.

◆ Árvores

São estruturas hierárquicas formadas por **nós**.

- O primeiro nó é a **raiz**.
- Cada nó pode ter **filhos**.
- É usada para representar hierarquias (como pastas em um computador).

Árvores Binárias

São árvores em que cada nó pode ter no máximo **dois filhos**: um à esquerda e um à direita.

Muito usadas em buscas e organização de dados (ex.: árvores de busca binária).

Exemplo: `Node raiz = new Node(10); raiz.Esquerda = new Node(5);
raiz.Direita = new Node(15); Console.WriteLine(raiz.Esquerda.Valor);`
→ o resultado é **5**, que é o valor do filho à esquerda da raiz.

Beleza, Kau 🙌 vou te dar um **resumão só em texto**, direto e sem código, dos pontos extras que podem cair além do que você já tem:

◆ Matrizes (Arrays Multidimensionais)

São estruturas em forma de tabela (linhas e colunas), diferentes dos jagged arrays, pois todas as linhas têm o mesmo tamanho. Muito usadas para representar grades, tabelas, jogos (como jogo da velha).

◆ List (Lista Genérica em C#)

É uma coleção dinâmica que pode aumentar ou diminuir de tamanho conforme necessário. Diferente do array fixo, a List permite inserções e remoções de forma mais flexível, mas ainda mantém a ideia de índices.

◆ Diferenças entre Estruturas

- **Array**: tamanho fixo, acesso rápido, mas difícil inserir/remover.
 - **List**: tamanho variável, fácil manipulação, mas pode gastar mais memória.
 - **LinkedList**: não ocupa memória contínua, facilita inserir/remover, mas acessar um elemento é mais lento.
 - **Pilha (Stack)**: segue o princípio LIFO, o último que entra é o primeiro que sai.
 - **Fila (Queue)**: segue o princípio FIFO, o primeiro que entra é o primeiro que sai.
-

◆ Complexidade Básica

É uma forma de medir eficiência.

- Procurar em um array → **$O(n)$** , pois pode ser necessário percorrer todos os elementos.
 - Procurar em uma árvore de busca binária → **$O(\log n)$** , pois descemos pelos níveis dividindo o problema.
 - Inserir ou remover em uma lista ligada → eficiente, mas encontrar o ponto certo para isso pode ser mais lento.
-

◆ Recursão e Casos de Erro

A recursão precisa de um **caso base**, que interrompe as chamadas.

Se não tiver caso base, a função nunca termina e gera erro de **estouro de pilha (StackOverflow)**.

◆ Percurso em Árvores Binárias

Existem três formas clássicas de percorrer os nós de uma árvore binária:

- **Pré-ordem:** visita primeiro a raiz, depois esquerda e direita.
- **Em ordem:** visita esquerda, raiz e direita (usada em árvores de busca, pois gera os valores em ordem crescente).
- **Pós-ordem:** visita primeiro os filhos, depois a raiz.

◆ Estruturas Lineares vs Não Lineares

- **Lineares:** arrays, listas, pilhas, filas. Os dados têm ordem única e sequencial.
 - **Não Lineares:** árvores e grafos. Os dados podem ter várias ramificações e conexões.
-

◆ Convenções de Nomeclatura

binarytreenode

Tudo minúsculo, sem separação.

Usado raramente, geralmente para nomes de arquivos simples ou comandos em sistemas que não diferenciam maiúsculas de minúsculas.

binaryTreeNode

Camel Case (camelCase).

Usado para nomes de variáveis e métodos em muitas linguagens (JavaScript, Java, C# para variáveis locais).

BinaryTreeNode

Pascal Case (PascalCase).

Usado para nomes de classes, structs, interfaces e enums em C#, Java e outras linguagens orientadas a objetos.

binary_tree_node

Snake Case (snake_case).

Usado em Python para variáveis e funções, ou em bancos de dados e arquivos de configuração.

BINARYTREENODE

Tudo maiúsculo.

Usado para constantes globais, especialmente em C, C++ e algumas convenções de C#.

Resumo prático:

- Variáveis/métodos → `binaryTreeNode` (camelCase)
 - Classes/tipos → `BinaryTreeNode` (PascalCase)
 - Constantes → `BINARYTREENODE` (UPPERCASE)
 - Python/SQL → `binary_tree_node` (snake_case)
-

♦ Recursão – Casos

Caso Base

É a condição que encerra a recursão.

Define o momento em que a função para de se chamar novamente, evitando loop infinito e estouro de pilha (StackOverflow).

Caso Recursivo

É a parte da função em que ela se chama novamente com um problema reduzido ou simplificado.

Ele deve sempre convergir para o caso base, garantindo que a recursão termine.
